

第 7 章

图像平滑处理

在尽量保留图像原有信息的情况下，过滤掉图像内部的噪声，这一过程称为对图像的平滑处理，所得的图像称为平滑图像。例如，图 7-1 是含有噪声的图像，在图像内存在噪声信息，我们通常会通过图像平滑处理等方式去除这些噪声信息。



图 7-1 含有噪声的图像

通过图像平滑处理，可以有效地过滤掉图像内的噪声信息。如图 7-2 所示是对图 7-1 进行图像平滑处理的结果，可以看到原有图像内含有的噪声信息被有效地过滤掉了。



图 7-2 平滑图像

图像平滑处理会对图像中与周围像素点的像素值差异较大的像素点进行处理，将其值调整为周围像素点像素值的近似值。例如，假设图 7-3 是一幅图像的像素点值。在图 7-3 中，大部分像素点值在[145,150]区间内，只有位于第 3 行第 3 列的像素点的值“29”不在这个范围内。



145	148	146	146	149
150	150	148	145	146
145	150	29	145	148
147	146	149	145	148
147	145	149	148	147

图 7-3 一幅图像的像素值示例

位于第 3 行第 3 列的像素点，与周围像素点值的大小存在明显差异。反映在图像上，该点周围的像素点都是灰度点，而该点的颜色较深，是一个黑色点，如图 7-4 所示。

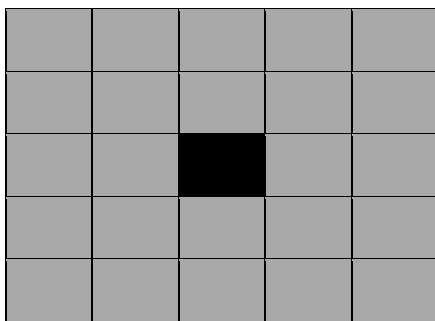


图 7-4 图像颜色示例

从图中可以看出，位于第 3 行第 3 列的颜色较深的点可能是噪声，需要将该点的值调整为周围像素值的近似值。如图 7-5 所示，是针对第 3 行第 3 列的像素点进行平滑处理的结果，平滑处理后，该点的像素值由 29 变为 148。

145	148	146	146	149
150	150	148	145	146
145	150	148	145	148
147	146	149	145	148
147	145	149	148	147

图 7-5 平滑处理像素值

这些像素点对应的图像如图 7-6 所示，可以看到，在对第 3 行第 3 列的像素点进行平滑处理后，图像内所有像素的颜色趋于一致。

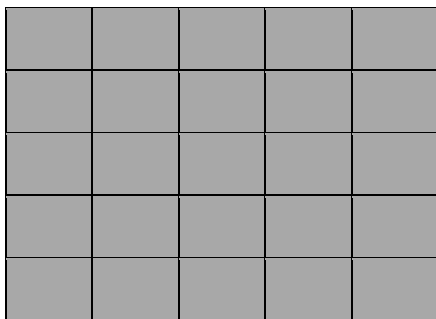


图 7-6 平滑图像

如果针对图像内的每一个像素点都进行上述平滑处理，就能够对整幅图像完成平滑处理，有效地去除图像内的噪声信息。

图像平滑处理的基本原理是，将噪声所在像素点的像素值处理为其周围临近像素点的值的近似值。取近似值的方式很多，本章主要介绍：

- 均值滤波
- 方框滤波
- 高斯滤波
- 中值滤波
- 双边滤波
- 2D 卷积（自定义滤波）

图像平滑处理对应的是英文 **Smoothing Images**。图像平滑处理通常伴随图像模糊操作，因此图像平滑处理有时也被称为图像模糊处理，图像模糊处理对应的英文是 **Blurring Images**。

图像滤波是图像处理和计算机视觉中最常用、最基本的操作。图像滤波允许在图像上进行各种各样的操作，因此有时我们也会把图像平滑处理称为图像滤波，图像滤波对应的英文是 **Images Filtering**。

在阅读文献时，针对图像平滑处理，我们可能会遇到各种不同的说法，因为不同的学者对于图像平滑处理的称呼可能不一样。所以，希望大家在学习时不要太纠结这些称呼的不同，而要将注意力集中在如何更好地理解图像算法、如何更好地处理图像上。

7.1 均值滤波

均值滤波是指用当前像素点周围 $N \cdot N$ 个像素值的均值来代替当前像素值。使用该方法遍历处理图像内的每一个像素点，即可完成整幅图像的均值滤波。

7.1.1 基本原理

例如，希望对图 7-7 中位于第 5 行第 5 列的像素点进行均值滤波。

23	158	140	115	131	87	131
238	0	67	16	247	14	220
199	197	25	106	156	159	173
94	149	40	107	5	71	171
210	163	198	226	223	156	159
107	222	37	68	193	157	110
255	42	72	250	41	75	184
77	150	17	248	197	147	150
218	235	106	128	65	197	202

图 7-7 一幅图像的像素值示例

在进行均值滤波时，首先要考虑需要对周围多少个像素点取平均值。通常情况下，我们会以当前像素点为中心，对行数和列数相等的一块区域内的所有像素点的像素值求平均。例如，在图 7-7 中，可以以当前像素点为中心，对周围 3×3 区域内所有像素点的像素值求平均，也可以对周围 5×5 区域内所有像素点的像素值求平均。

当前像素点的位置为第 5 行第 5 列，我们对其周围 5×5 区域内的像素值求平均，计算方法为：

$$\begin{aligned}
 \text{新值} &= [(197+25+106+156+159)+ \\
 &\quad (149+40+107+5+71)+ \\
 &\quad (163+198+226+223+156)+ \\
 &\quad (222+37+68+193+157)+ \\
 &\quad (42+72+250+41+75)]/25 \\
 &= 126
 \end{aligned}$$

计算完成后得到 126，我们将 126 作为当前像素点均值滤波后的像素值。我们针对图 7-7 中的每一个像素点计算其周围 5×5 区域内的像素值均值，并将其作为当前像素点的新值，即可得到当前图像的均值滤波结果。

当然，图像的边界点并不存在 5×5 邻域区域。例如，左上角第 1 行第 1 列上的像素点，其像素值为 23，如果以其为中心点取周围 5×5 邻域，则 5×5 邻域中的部分区域位于图像外部。图像外部是没有像素点和像素值的，显然是无法计算该点的 5×5 邻域均值的。

针对边缘像素点，可以只取图像内存在的周围邻域点的像素值均值。如图 7-8 所示，计算左上角的均值滤波结果时，仅取图中灰色背景的 3×3 邻域内的像素值的平均值。

23	158	140	115	131	87	131
238	0	67	16	247	14	220
199	197	25	106	156	159	173
94	149	40	107	5	71	171
210	163	198	226	223	156	159
107	222	37	68	193	157	110
255	42	72	250	41	75	184
77	150	17	248	197	147	150
218	235	106	128	65	197	202

图 7-8 边界点的处理

在图 7-8 中，对于左上角（第 1 行第 1 列）的像素点，我们取第 1~3 列与第 1~3 行交汇处所包含的 3×3 邻域内的像素点的像素值均值。因此，当前像素点的均值滤波计算方法为：

$$\begin{aligned} \text{新值} &= [(23+158+140)+ \\ &\quad (238+0+67)+ \\ &\quad (199+197+25)]/9 \\ &= 116 \end{aligned}$$

计算完成后得到 116，将该值作为该点的滤波结果即可。

除此以外，还可以扩展当前图像的周围像素点。例如，将当前 9×7 大小的图像扩展为 13×11 大小的图像，如图 7-9 所示。

		23	158	140	115	131	87	131		
		238	0	67	16	247	14	220		
		199	197	25	106	156	159	173		
		94	149	40	107	5	71	171		
		210	163	198	226	223	156	159		
		107	222	37	68	193	157	110		
		255	42	72	250	41	75	184		
		77	150	17	248	197	147	150		
		218	235	106	128	65	197	202		

图 7-9 扩展边缘



完成图像边缘扩展后,可以在新增的行列内填充不同的像素值。在此基础上,再针对 9×7 的原始图像计算其 5×5 邻域内像素点的像素值均值。OpenCV 提供了多种边界处理方式,我们可以根据实际需要选用不同的边界处理模式。

针对图像中第 5 行第 5 列的像素点,其运算过程相当于与一个内部值都是 $1/25$ 的 5×5 矩阵进行相乘计算,从而得到均值滤波的结果 126,其对应的关系如图 7-10 所示。

23	158	140	115	131	87	131
238	0	67	16	247	14	220
199	197	25	106	156	159	173
94	149	40	107	5	71	171
210	163	198	226	223	156	159
107	222	37	68	193	157	110
255	42	72	250	41	75	184
77	150	17	248	197	147	150
218	235	106	128	65	197	202

×

1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25

=

			126			

图 7-10 针对第 5 行第 5 列像素点均值滤波的运算示意图

根据上述运算,针对每一个像素点,都是与一个内部值均为 $1/25$ 的 5×5 矩阵相乘,得到均值滤波的计算结果,如图 7-11 所示。

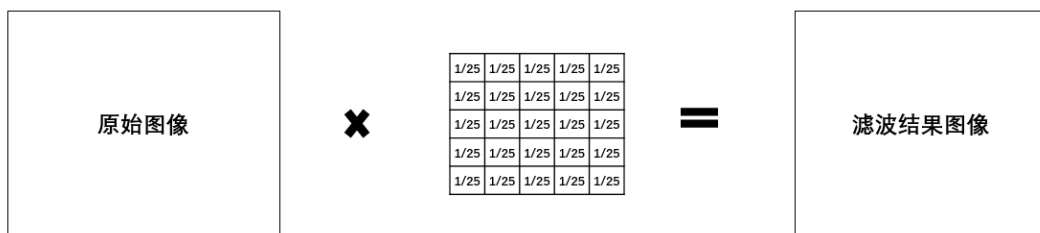


图 7-11 针对每一个像素点均值滤波的运算示意图

将使用的 5×5 矩阵一般化,可以得到如图 7-12 所示的结果。

1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25

→

$$K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

图 7-12 将矩阵一般化

在 OpenCV 中,图 7-12 右侧的矩阵被称为卷积核,其一般形式为:

$$K = \frac{1}{M \cdot N} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \end{bmatrix}$$

式中, M 和 N 分别对应高度和宽度。一般情况下, M 和 N 是相等的,例如比较常用的 3×3 、 5×5 、 7×7 等。如果 M 和 N 的值越大,参与运算的像素点数量就越多,图像失真越严重。



7.1.2 函数语法

在 OpenCV 中，实现均值滤波的函数是 `cv2.blur()`，其语法格式为：

```
dst = cv2.blur( src, ksize, anchor, borderType )
```

式中：

- `dst` 是返回值，表示进行均值滤波后得到的处理结果。
- `src` 是需要处理的图像，即原始图像。它可以有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `ksize` 是滤波核的大小。滤波核大小是指在均值处理过程中，其邻域图像的高度和宽度。例如，其值可以为 `(5, 5)`，表示以 5×5 大小的邻域均值作为图像均值滤波处理的结果，如下式所示。

$$K = \frac{1}{5 \times 5} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

- `anchor` 是锚点，其默认值是 `(-1, -1)`，表示当前计算均值的点位于核的中心点位置。该值使用默认值即可，在特殊情况下可以指定不同的点作为锚点。
- `borderType` 是边界样式，该值决定了以何种方式处理边界，其值如表 7-1 所示。一般情况下不需要考虑该值的取值，直接采用默认值即可。

表 7-1 borderType 值

类型	说明
<code>cv2.BORDER_CONSTANT</code>	iiiiii abcdefgh iiiiiii, 特定值 i
<code>cv2.BORDER_REPLICATE</code>	aaaaaa abcdefgh hhhhhhh
<code>cv2.BORDER_REFLECT</code>	fedcba abcdefgh hgfedcb
<code>cv2.BORDER_WRAP</code>	cdefgh abcdefgh abcdefg
<code>cv2.BORDER_REFLECT_101</code>	gfedcb abcdefgh gfedcba
<code>cv2.BORDER_TRANSPARENT</code>	uvwxyz abcdefgh ijklmno
<code>cv2.BORDER_REFLECT101</code>	与 <code>BORDER_REFLECT_101</code> 相同
<code>cv2.BORDER_DEFAULT</code>	与 <code>BORDER_REFLECT_101</code> 相同
<code>cv2.BORDER_ISOLATED</code>	不考虑 ROI (Region of Interest, 感兴趣区域) 以外的区域

通常情况下，使用均值滤波函数时，对于锚点 `anchor` 和边界样式 `borderType`，直接采用其默认值即可。因此，函数 `cv2.blur()` 的一般形式为：

```
dst = cv2.blur( src, ksize, )
```

7.1.3 程序示例

【例 7.1】读取一幅噪声图像，使用函数 `cv2.blur()` 对图像进行均值滤波处理，得到去噪图像，并显示原始图像和去噪图像。

根据题目要求，编写代码如下：


```
import cv2
o=cv2.imread("image\\lenaNoise.png")    #读取待处理图像
r=cv2.blur(o,(5,5))                      #使用 blur 函数处理
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序后，会分别显示噪声图像（左图）和去噪图像（右图），如图 7-13 所示。

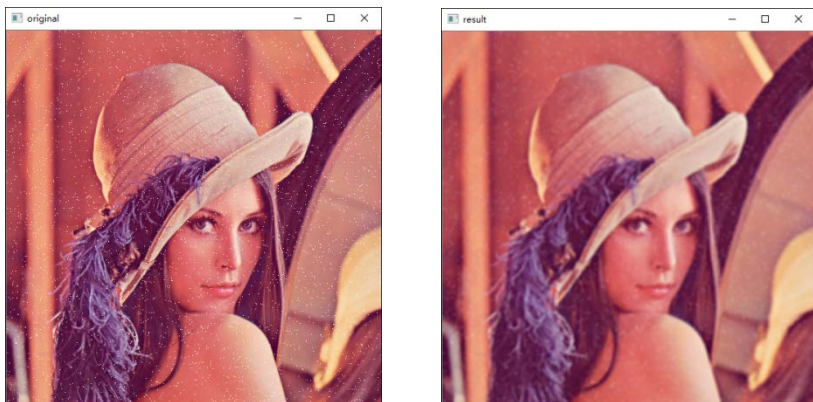


图 7-13 中值滤波示例

【例 7.2】针对噪声图像，使用不同大小的卷积核对其进行均值滤波，并显示均值滤波的情况。

根据题目要求，调整设置函数 `cv2.blur()` 中的 `ksize` 参数，分别将卷积核设置为 5×5 大小和 30×30 大小，对比均值滤波的结果。所编写的代码为：

```
import cv2
o=cv2.imread("image\\noise.bmp")
r5=cv2.blur(o,(5,5))
r30=cv2.blur(o,(30,30))
cv2.imshow("original",o)
cv2.imshow("result5",r5)
cv2.imshow("result30",r30)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，分别显示原始图像（图(a)）、使用 5×5 卷积核进行均值滤波处理结果图像（图(b)）、使用 30×30 卷积核进行均值滤波处理结果图像（图(c)），如图 7-14 所示。

从图中可以看出，使用 5×5 的卷积核进行滤波处理时，图像的失真不明显；而使用 30×30 的卷积核进行滤波处理时，图像的失真情况较明显。

卷积核越大，参与到均值运算中的像素就会越多，即当前点计算的是更多点的像素值的平均值。因此，卷积核越大，去噪效果越好，当然花费的计算时间也会越长，同时让图像失真越严重。在实际处理中，要在失真和去噪效果之间取得平衡，选取合适大小的卷积核。

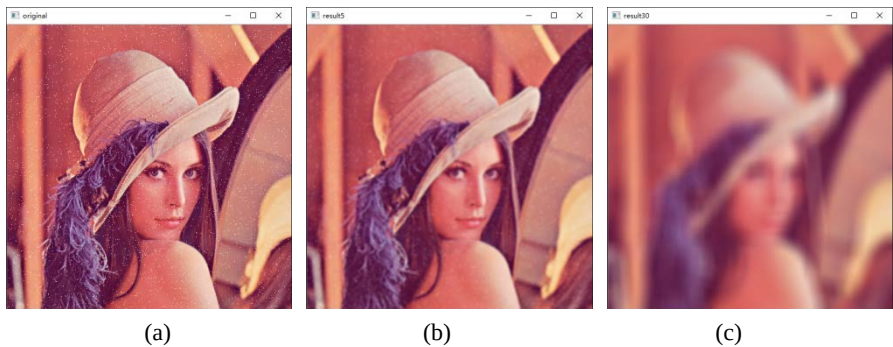


图 7-14 不同大小卷积核的均值滤波结果

7.2 方框滤波

OpenCV 还提供了方框滤波方式，与均值滤波的不同在于，方框滤波不会计算像素均值。在均值滤波中，滤波结果的像素值是任意一个点的邻域平均值，等于各邻域像素值之和除以邻域面积。而在方框滤波中，可以自由选择是否对均值滤波的结果进行归一化，即可以自由选择滤波结果是邻域像素值之和的平均值，还是邻域像素值之和。

7.2.1 基本原理

我们以 5×5 的邻域为例，在进行方框滤波时，如果计算的是邻域像素值的均值，则滤波关系如图 7-15 所示。



图 7-15 方框滤波关系示例 1

仍然以 5×5 的邻域为例，在进行方框滤波时，如果计算的是邻域像素值之和，则滤波关系如图 7-16 所示。

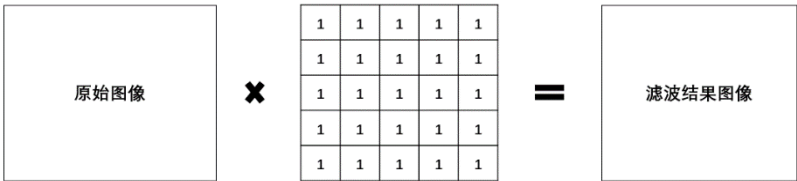


图 7-16 方框滤波关系示例 2

根据上述关系，如果计算的是邻域像素值的均值，则使用的卷积核为：

$$K = \frac{1}{5 \times 5} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

如果计算的是邻域像素值之和，则使用的卷积核为：

$$K = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

7.2.2 函数语法

在 OpenCV 中，实现方框滤波的函数是 `cv2.boxFilter()`，其语法格式为：

`dst = cv2.boxFilter( src, ddepth, ksize, anchor, normalize, borderType )`

式中：

- `dst` 是返回值，表示进行方框滤波后得到的处理结果。
- `src` 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `ddepth` 是处理结果图像的图像深度，一般使用 -1 表示与原始图像使用相同的图像深度。
- `ksize` 是滤波核的大小。滤波核大小是指在滤波处理过程中所选择的邻域图像的高度和宽度。例如，滤波核的值可以为 (3,3)，表示以 3×3 大小的邻域均值作为图像均值滤波处理的结果，如下式所示。

$$K = \frac{1}{3 \times 3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- `anchor` 是锚点，其默认值是 (-1, -1)，表示当前计算均值的点位于核的中心点位置。该值使用默认值即可，在特殊情况下可以指定不同的点作为锚点。
- `normalize` 表示在滤波时是否进行归一化（这里指将计算结果规范化为当前像素值范围内的值）处理，该参数是一个逻辑值，可能为真（值为 1）或假（值为 0）。
 - 当参数 `normalize=1` 时，表示要进行归一化处理，要用邻域像素值的和除以面积。
 - 当参数 `normalize=0` 时，表示不需要进行归一化处理，直接使用邻域像素值的和。

通常情况下，针对方框滤波，卷积核可以表示为：

$$K = \frac{1}{\alpha} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 \end{bmatrix}$$

上述对应关系为：



$$\alpha = \begin{cases} \frac{1}{\text{width} \cdot \text{height}}, & \text{normalize} = 1 \\ 1, & \text{normalize} = 0 \end{cases}$$

例如，针对 5×5 邻域，当参数 `normalize=1` 时，要进行归一化处理，此时计算的就是均值滤波。这种情况下，函数 `cv2.boxFilter()` 和函数 `cv2.blur()` 的作用是一样的。此时，对应的卷积核为：

$$K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

同样针对 5×5 邻域，当参数 `normalize=0` 时，不进行归一化处理，此时滤波计算的是邻域像素值之和，使用的卷积核是：

$$K = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

当 `normalize=0` 时，因为不进行归一化处理，因此滤波得到的值很可能超过当前像素值范围的最大值，从而被截断为最大值。这样，就会得到一幅纯白色的图像。

- `borderType` 是边界样式，该值决定了以何种方式处理边界。

通常情况下，在使用方框滤波函数时，对于参数 `anchor`、`normalize` 和 `borderType`，直接采用其默认值即可。因此，函数 `cv2.boxFilter()` 的常用形式为：

```
dst = cv2.boxFilter( src, ddepth, ksize )
```

7.2.3 程序示例

【例 7.3】 针对噪声图像，对其进行方框滤波，显示滤波结果。

根据题目要求，使用方框滤波函数 `cv2.boxFilter()` 对原始图像进行滤波，设计代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.boxFilter(o, -1, (5,5))
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，方框滤波函数对 `normalize` 参数使用了默认值。在默认情况下，该值为 1，表示要进行归一化处理。也就是说，本例中使用的是 `normalize` 为默认值 `True` 的 `cv2.boxFilter()` 函数，此时它和函数 `cv2.blur()` 的滤波结果是完全相同的。如图 7-17 所示，左图是原始图像，右图是方框滤波结果图像。

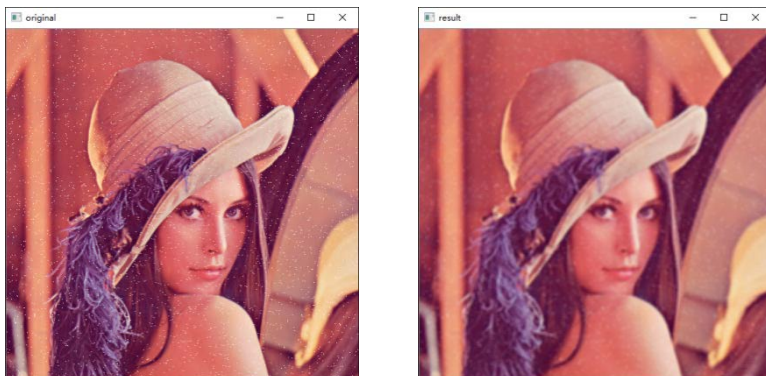


图 7-17 方框滤波示例

在本例中，方框滤波语句 `r=cv2.boxFilter(o,-1,(5,5))` 使用了参数 `normalize` 的默认值，相当于省略了 `normalize=1`，因此，该语句与 `r=cv2.boxFilter(o,-1,(5,5),normalize=1)` 是等效的。当然，此时该语句与均值滤波语句 `r5=cv2.blur(o,(5,5))` 也是等效的。

【例 7.4】针对噪声图像，在方框滤波函数 `cv2.boxFilter()` 内将参数 `normalize` 的值设置为 0，显示滤波处理结果。

根据题目要求，将参数 `normalize` 设置为 0，编写程序代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.boxFilter(o, -1, (5,5),normalize=0)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，没有对图像进行归一化处理。在进行滤波时，计算的是 5×5 邻域的像素值之和，图像的像素值基本都会超过当前像素值的最大值 255。因此，最后得到的图像接近纯白色，部分点处有颜色。部分点有颜色是因为这些点周边邻域的像素值均较小，邻域像素值在相加后仍然小于 255。

此时的图像滤波结果如图 7-18 所示，左图是原始图像，右图是方框滤波后得到的处理结果。

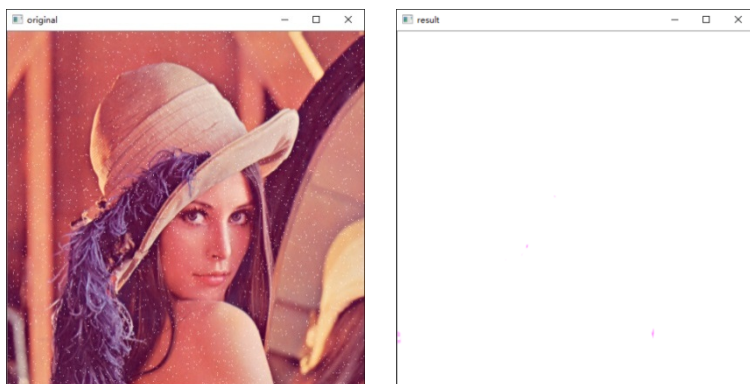


图 7-18 【例 7.4】对应的方框滤波结果

【例 7.5】 针对噪声图像，使用方框滤波函数 `cv2.boxFilter()` 去噪，将参数 `normalize` 的值设置为 0，将卷积核的大小设置为 2×2 ，显示滤波结果。

根据题目要求，编写程序代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.boxFilter(o, -1, (2,2),normalize=0)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，卷积核大小为 2×2 ，参数 `normalize=0`。因此，本例中方框滤波计算的是 2×2 邻域的像素值之和，四个像素值的和不一定大于 255，因此在计算结果图像中有部分像素点不是白色。如图 7-19 所示，左图是原始图像，右图是方框滤波处理结果。

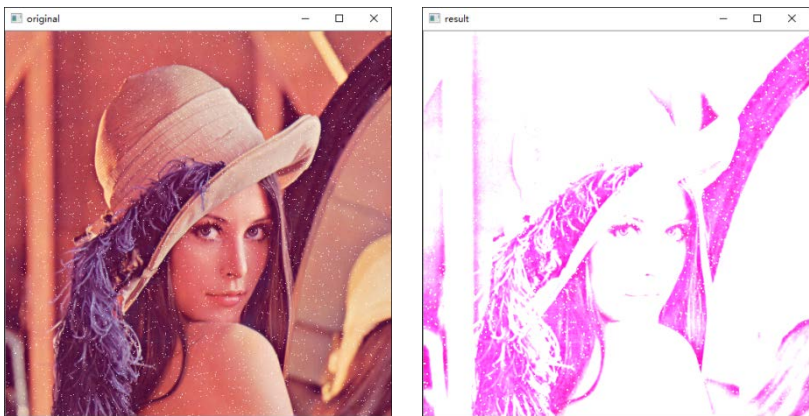


图 7-19 【例 7.5】对应的方框滤波结果

7.3 高斯滤波

在进行均值滤波和方框滤波时，其邻域内每个像素的权重是相等的。在高斯滤波中，会将中心点的权重值加大，远离中心点的权重值减小，在此基础上计算邻域内各个像素值不同权重的和。

7.3.1 基本原理

在高斯滤波中，卷积核中的值不再都是 1。例如，一个 3×3 的卷积核可能如图 7-20 所示：

1	2	1
2	8	2
1	2	1

图 7-20 高斯滤波卷积核示例

在图 7-21 中, 针对最左侧的图像内第 4 行第 3 列位置上像素值为 226 的像素点进行高斯卷积, 其运算规则为将该点邻域内的像素点按照不同的权重计算和。

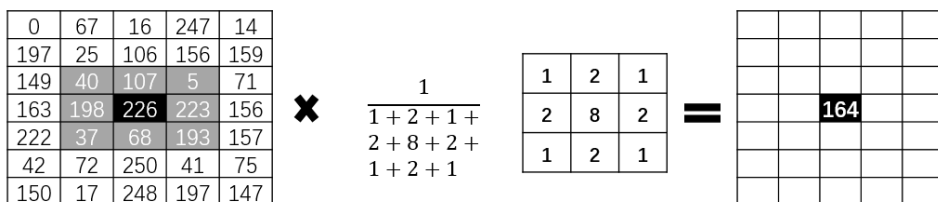


图 7-21 高斯卷积示例

在实际计算时, 使用的卷积核如图 7-22 中的卷积核所示。

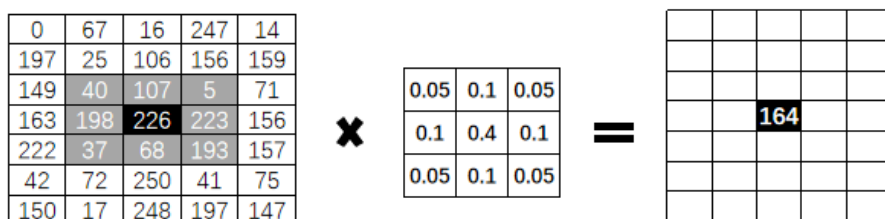


图 7-22 实际计算中的卷积核

使用图 7-22 中的卷积核, 针对第 4 行第 3 列位置上的像素值为 226 的像素点进行高斯滤波处理, 计算方式为:

$$\begin{aligned}
 \text{新值} &= (40 \times 0.05 + 107 \times 0.1 + 5 \times 0.05) \\
 &\quad + (198 \times 0.1 + 226 \times 0.4 + 223 \times 0.1) \\
 &\quad + (37 \times 0.05 + 68 \times 0.1 + 193 \times 0.05) \\
 &= 164
 \end{aligned}$$

在实际使用中, 高斯滤波使用的可能是不同大小的卷积核。例如, 图 7-23 中分别是 3×3、5×5、7×7 大小的卷积核。在高斯滤波中, 核的宽度和高度可以不相同, 但是它们都必须都是奇数。

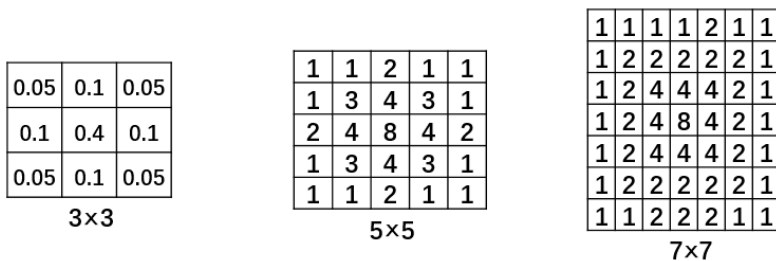


图 7-23 不同大小的卷积核

每一种尺寸的卷积核都可以有多种不同形式的权重比例。例如, 同样是 5×5 的卷积核, 可能是图 7-24 中所示的两种不同的权重比。

$$\frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix} \quad \frac{1}{159} \begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

图 7-24 同一尺寸的卷积核可以有不同权重比

在不同的资料中，卷积核有多种不同的表示方式。它们可能如图 7-23 所示写在一个表格内，也可能如图 7-24 所示写在一个矩阵内。

在实际计算中，卷积核是归一化处理的，这种处理可以表示为图 7-23 最左侧的小数形式的卷积核，也可以表示为如图 7-24 所示的分数形式。也要注意，在一些资料中，给出的卷积核并没有进行归一化，这时的卷积核可能表示为图 7-23 中间和右侧所示的卷积核，这样的卷积核是为了说明问题用的，实际使用时往往需要进行归一化。严格来讲，使用没有进行归一化处理的卷积核进行滤波，得到的结果往往是错误的。

7.3.2 函数语法

在 OpenCV 中，实现高斯滤波的函数是 `cv2.GaussianBlur()`，该函数的语法格式是：

`dst = cv2.GaussianBlur( src, ksize, sigmaX, sigmaY, borderType )`

式中：

- `dst` 是返回值，表示进行高斯滤波后得到的处理结果。
- `src` 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `ksize` 是滤波核的大小。滤波核大小是指在滤波处理过程中其邻域图像的高度和宽度。需要注意，滤波核的值必须是奇数。
- `sigmaX` 是卷积核在水平方向上（X 轴方向）的标准差，其控制的是权重比例。例如，图 7-25 中是不同的 `sigmaX` 决定的卷积核，它们在水平方向上的标准差不同。

1	1	2	1	1
1	2	4	2	1
2	4	8	4	2
1	4	4	2	1
1	1	2	1	1

0	0	1	0	0
0	1	2	1	0
1	2	3	2	1
0	1	2	1	0
0	0	1	0	0

0	3	6	3	0
1	4	7	4	1
2	5	8	5	2
1	4	7	4	1
0	3	6	3	0

图 7-25 不同的 `sigmaX` 决定的卷积核

- `sigmaY` 是卷积核在垂直方向上（Y 轴方向）的标准差。如果将该值设置为 0，则只采用 `sigmaX` 的值；如果 `sigmaX` 和 `sigmaY` 都是 0，则通过 `ksize.width` 和 `ksize.height` 计算得到。

其中：

- `sigmaX = 0.3 × [(ksize.width - 1) × 0.5 - 1] + 0.8`
- `sigmaY = 0.3 × [(ksize.height - 1) × 0.5 - 1] + 0.8`
- `borderType` 是边界样式，该值决定了以何种方式处理边界。一般情况下，不需要考虑该

值，直接采用默认值即可。

在该函数中，`sigmaY` 和 `borderType` 是可选参数。`sigmaX` 是必选参数，但是可以将该参数设置为 0，让函数自己去计算 `sigmaX` 的具体值。

官方文档建议显式地指定 `ksize`、`sigmaX` 和 `sigmaY` 三个参数的值，以避免将来函数修改后可能造成的语法错误。当然，在实际处理中，可以显式指定 `sigmaX` 和 `sigmaY` 为默认值 0。因此，函数 `cv2.GaussianBlur()` 的常用形式为：

```
dst = cv2.GaussianBlur( src, ksize, 0, 0 )
```

7.3.3 程序示例

【例 7.6】 对噪声图像进行高斯滤波，显示滤波的结果。

根据题目要求，采用函数 `cv2.GaussianBlur()` 实现高斯滤波，编写程序代码为：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.GaussianBlur(o,(5,5),0,0)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序后，结果如图 7-26 所示，其中左图是原始图像，右图是高斯滤波后的处理结果图像。

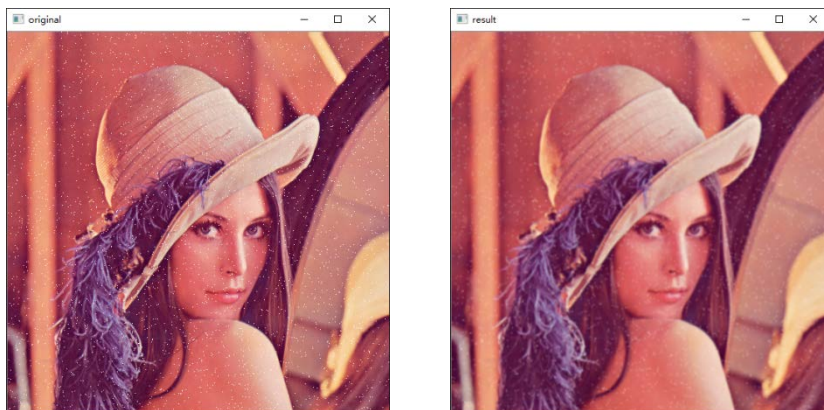


图 7-26 高斯滤波示例

7.4 中值滤波

中值滤波与前面介绍的滤波方式不同，不再采用加权求均值的方式计算滤波结果。它用邻域内所有像素值的中间值来替代当前像素点的像素值。

7.4.1 基本原理

中值滤波会取当前像素点及其周围临近像素点（一共有奇数个像素点）的像素值，将这些像素值排序，然后将位于中间位置的像素值作为当前像素点的像素值。

例如，针对图 7-27 中第 4 行第 4 列的像素点，计算它的中值滤波值。

55	58	22	55	22	60	168	162	232
123	17	66	33	77	68	14	74	67
47	22	97	95	94	25	14	5	76
68	66	93	78	90	171	82	78	65
69	99	66	91	101	200	192	59	74
98	88	88	45	36	119	47	28	5
88	158	3	88	69	211	234	192	120
77	148	25	45	77	173	226	146	213
42	125	135	58	44	51	79	66	3

图 7-27 一幅图像的像素值示例

将其邻域设置为 3×3 大小，对其 3×3 邻域内像素点的像素值进行排序（升序降序均可），按升序排序后得到序列值为：[66,78,90,91,93,94,95,97,101]。在该序列中，处于中心位置（也叫中心点或中值点）的值是“93”，因此用该值替换原来的像素值 78，作为当前点的新像素值，处理结果如图 7-28 所示。

55	58	22	55	22	60	168	162	232
123	17	66	33	77	68	14	74	67
47	22	97	95	94	25	14	5	76
68	66	93	93	90	171	82	78	65
69	99	66	91	101	200	192	59	74
98	88	88	45	36	119	47	28	5
88	158	3	88	69	211	234	192	120
77	148	25	45	77	173	226	146	213
42	125	135	58	44	51	79	66	3

图 7-28 中值滤波处理结果

7.4.2 函数语法

在 OpenCV 中，实现中值滤波的函数是 `cv2.medianBlur()`，其语法格式如下：

```
dst = cv2.medianBlur( src, ksize)
```

式中：

- `dst` 是返回值，表示进行中值滤波后得到的处理结果。
- `src` 是需要处理的图像，即源图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `ksize` 是滤波核的大小。滤波核大小是指在滤波处理过程中其邻域图像的高度和宽度。需要注意，核大小必须是比 1 大的奇数，比如 3、5、7 等。

7.4.3 程序示例

【例 7.7】 针对噪声图像，对其进行中值滤波，显示滤波的结果。

根据题目要求，采用函数 `cv2.medianBlur()` 实现中值滤波，编写程序代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.medianBlur(o,3)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序后，结果如图 7-29 所示，其中左图是原始图像，右图是中值滤波后的处理结果图像。

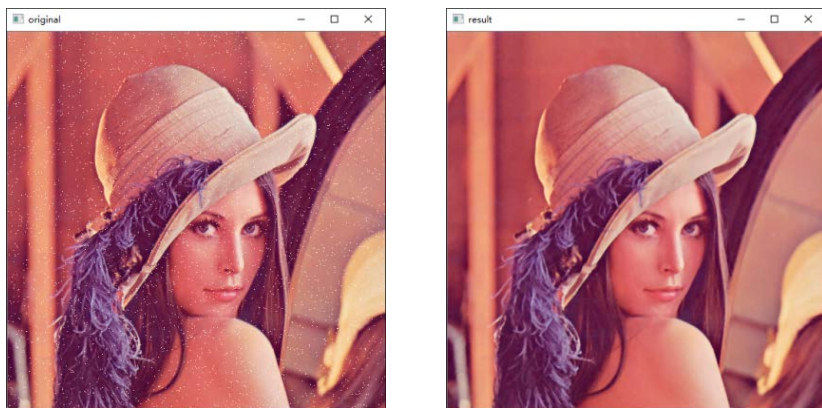


图 7-29 中值滤波示例

从图中可以看到，由于没有进行均值处理，中值滤波不存在均值滤波等滤波方式带来的细节模糊问题。在中值滤波处理中，噪声成分很难被选上，所以可以在几乎不影响原有图像的情况下去除全部噪声。但是由于需要进行排序等操作，中值滤波需要的运算量较大。

7.5 双边滤波

双边滤波是综合考虑空间信息和色彩信息的滤波方式，在滤波过程中能够有效地保护图像内的边缘信息。

7.5.1 基本原理

前述滤波方式基本都只考虑了空间的权重信息，这种情况计算起来比较方便，但是在边缘信息的处理上存在较大的问题。

例如，在图 7-30 中，图像左侧是黑色，右侧是白色，中间是很明显的边缘。

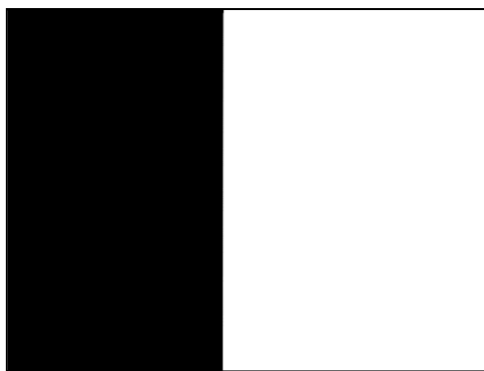


图 7-30 边缘示例

在均值滤波、方框滤波、高斯滤波中，都会计算边缘上各个像素点的加权平均值，从而模糊边缘信息。如图 7-31 所示是高斯滤波处理的结果图像。



图 7-31 高斯滤波处理后导致边缘模糊

从图 7-31 可以看到，经过高斯滤波处理后，边缘信息变得很模糊，均值滤波处理也会造成类似的问题。边界模糊是滤波处理过程中对邻域像素取均值所造成的结果，上述滤波处理过程单纯地考虑空间信息，造成了边界模糊和部分信息的丢失。

双边滤波在计算某一个像素点的新值时，不仅考虑距离信息（距离越远，权重越小），还考虑色彩信息（色彩差别越大，权重越小）。双边滤波综合考虑距离和色彩的权重结果，既能有效地去除噪声，又能够较好地保护边缘信息。

在双边滤波中，当处在边缘时，与当前点色彩相近的像素点（颜色距离很近）会被给予较大的权重值；而与当前色彩差别较大的像素点（颜色距离很远）会被给予较小的权重值（极端情况下权重可能为 0，直接忽略该点），这样就保护了边缘信息。

例如，在图 7-32 中：

- 图(a)是原始图像，左侧区域是白色（像素值为 255），右侧区域是黑色（像素值为 0）。
- 图(b)是进行均值滤波的可能结果。在进行均值滤波时，仅仅考虑空间信息，此时左右两侧的像素的处理结果是综合考虑周边元素像素值，并对它们取均值得到的。
- 图(c)是进行双边滤波的可能结果。在进行双边滤波时，不仅考虑空间信息，还考虑色彩差别信息。

255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0
255	0	128	128	255	0

(a)
(b)
(c)

图 7-32 滤波示例

在双边滤波中，在计算左侧白色区域边缘点的滤波结果时：

- 对于白色的点，给予的权重较大。
- 对于黑色的点，由于色彩差异较大，颜色距离很远（注意，不是像素点之间的物理距离，而是颜色值的距离。像素点的值分别是 0 和 255，差别很大，所以说它们颜色距离很远），因此可以将它们的权重设置为 0。

这样，在计算左侧白色边缘滤波结果时，得到的仍然是白色。因此，双边滤波后，左侧边缘得到保留。

在计算右侧黑色区域边缘点的滤波结果时：

- 对于黑色的点，给予的权重较大。
- 对于白色的点，由于色彩差异较大，颜色距离很远，因此可以将它们的权重设置为 0。

这样，在计算右侧黑色边缘滤波结果时，得到的仍然是黑色。因此，双边滤波后，左侧边缘得到保留。



7.5.2 函数语法

在 OpenCV 中，实现双边滤波的函数是 `cv2.bilateralFilter()`，该函数的语法是：

```
dst = cv2.bilateralFilter( src, d, sigmaColor, sigmaSpace, borderType )
```

式中：

- `dst` 是返回值，表示进行双边滤波后得到的处理结果。
- `src` 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `d` 是在滤波时选取的空间距离参数，这里表示以当前像素点为中心点的直径。如果该值为非正数，则会自动从参数 `sigmaSpace` 计算得到。如果滤波空间较大 ($d > 5$)，则速度较慢。因此，在实时应用中，推荐 $d=5$ 。对于较大噪声的离线滤波，可以选择 $d=9$ 。
- `sigmaColor` 是滤波处理时选取的颜色差值范围，该值决定了周围哪些像素点能够参与到滤波中来。与当前像素点的像素值差值小于 `sigmaColor` 的像素点，能够参与到当前的滤波中。该值越大，就说明周围有越多的像素点可以参与到运算中。该值为 0 时，滤波失去意义；该值为 255 时，指定直径内的所有点都能够参与运算。
- `sigmaSpace` 是坐标空间中的 `sigma` 值。它的值越大，说明有越多的点能够参与到滤波计算中来。当 $d > 0$ 时，无论 `sigmaSpace` 的值如何， d 都指定邻域大小；否则， d 与 `sigmaSpace` 的值成比例。
- `borderType` 是边界样式，该值决定了以何种方式处理边界。一般情况下，不需要考虑该值，直接采用默认值即可。

为了简单起见，可以将两个 `sigma` (`sigmaColor` 和 `sigmaSpace`) 值设置为相同的。如果它们的值比较小（例如小于 10），滤波的效果将不太明显；如果它们的值较大（例如大于 150），则滤波效果会比较明显，会产生卡通效果。

在函数 `cv2.bilateralFilter()` 中，参数 `borderType` 是可选参数，其余参数全部为必选参数。

7.5.3 程序示例

【例 7.8】 针对噪声图像，对其进行双边滤波，显示滤波的结果。

根据题目要求，使用双边滤波函数 `cv2.bilateralFilter()` 对原始图像进行滤波，设计代码如下：

```
import cv2
o=cv2.imread("image\\lenaNoise.png")
r=cv2.bilateralFilter(o,25,100,100)
cv2.imshow("original",o)
cv2.imshow("result",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 7-33 所示，其中左图是原始图像，右图是双边滤波的结果图像。从图中可以看出，双边滤波去除噪声的效果并不好。

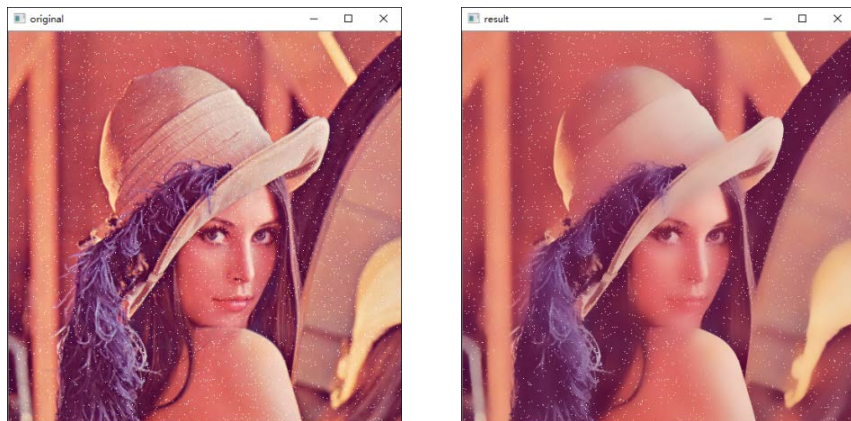


图 7-33 双边滤波示例

双边滤波的优势体现在对于边缘信息的处理上,下面通过一个例题来展示不同形式的滤波在边缘处理效果上的差异。

【例 7.9】针对噪声图像,分别对其进行高斯滤波和双边滤波,比较不同滤波方式对边缘的处理结果是否相同。

根据题目要求,分别使用高斯滤波函数 `cv2.GaussianBlur()` 和双边滤波函数 `cv2.bilateralFilter()`,对原始图像进行滤波,设计代码如下:

```
import cv2
o=cv2.imread("image\\bilTest.bmp")
g=r=cv2.GaussianBlur(o,(55,55),0,0)
b=cv2.bilateralFilter(o,55,100,100)
cv2.imshow("original",o)
cv2.imshow("Gaussian",g)
cv2.imshow("bilateral",b)
cv2.waitKey()
cv2.destroyAllWindows()
```

程序运行结果如图 7-34 所示,其中:

- 图(a)是原始图像。
- 图(b)是高斯滤波处理结果。
- 图(c)是双边滤波处理结果。

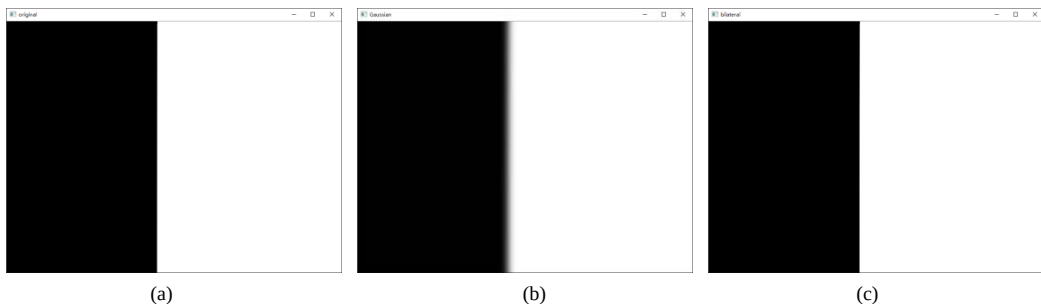


图 7-34 不同滤波方式比较

从图中可以看到，经过高斯滤波的边缘被模糊虚化了，而经过双边滤波的边缘得到了较好的保留。

7.6 2D 卷积

OpenCV 提供了多种滤波方式，来实现平滑图像的效果，例如均值滤波、方框滤波、高斯滤波、中值滤波等。大多数滤波方式所使用的卷积核都具有一定的灵活性，能够方便地设置卷积核的大小和数值。但是，我们有时希望使用特定的卷积核实现卷积操作，例如使用如下卷积核进行卷积操作。

$$\frac{1}{426} \begin{bmatrix} 6 & 6 & 6 & 6 & 6 \\ 6 & 33 & 33 & 33 & 6 \\ 6 & 33 & 66 & 33 & 6 \\ 6 & 33 & 33 & 33 & 6 \\ 6 & 6 & 6 & 6 & 6 \end{bmatrix} \quad \frac{1}{201} \begin{bmatrix} 5 & 5 & 5 & 5 & 5 \\ 5 & 12 & 12 & 12 & 5 \\ 5 & 12 & 15 & 12 & 5 \\ 5 & 12 & 12 & 12 & 5 \\ 5 & 5 & 5 & 5 & 5 \end{bmatrix}$$

前面介绍过的滤波函数都无法将卷积核确定为上述形式，这时要使用 OpenCV 的自定义卷积函数。

在 OpenCV 中，允许用户自定义卷积核实现卷积操作，使用自定义卷积核实现卷积操作的函数是 `cv2.filter2D()`，其语法格式为：

```
dst = cv2.filter2D( src, ddepth, kernel, anchor, delta, borderType )
```

式中：

- `dst` 是返回值，表示进行方框滤波后得到的处理结果。
- `src` 是需要处理的图像，即原始图像。它能够有任意数量的通道，并能对各个通道独立处理。图像深度应该是 `CV_8U`、`CV_16U`、`CV_16S`、`CV_32F` 或者 `CV_64F` 中的一种。
- `ddepth` 是处理结果图像的图像深度，一般使用 `-1` 表示与原始图像使用相同的图像深度。
- `kernel` 是卷积核，是一个单通道的数组。如果想在处理彩色图像时，让每个通道使用不同的核，则必须将彩色图像分解后使用不同的核完成操作。
- `anchor` 是锚点，其默认值是 `(-1, -1)`，表示当前计算均值的点位于核的中心点位置。该值使用默认值即可，在特殊情况下可以指定不同的点作为锚点。
- `delta` 是修正值，它是可选项。如果该值存在，会在基础滤波的结果上加上该值作为最终的滤波处理结果。
- `borderType` 是边界样式，该值决定了以何种情况处理边界，通常使用默认值即可。

在通常情况下，使用滤波函数 `cv2.filter2D()` 时，对于参数锚点 `anchor`、修正值 `delta`、边界样式 `borderType`，直接采用其默认值即可。因此，函数 `cv2.filter2D()` 的常用形式为：

```
dst = cv2.filter2D( src, ddepth, kernel )
```

【例 7.10】 自定义一个卷积核，通过函数 `cv2.filter2D()` 应用该卷积核对图像进行滤波操作，并显示滤波结果。

根据题目要求，设计一个 9×9 大小的卷积核，让卷积核内所有权重值相等，如下所示：

$$\frac{1}{81} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

借助 numpy 库中的 ones() 函数即可创建该卷积核，具体的语句为：

```
kernel = np.ones((9,9),np.float32)/81
```

综上所述，程序设计代码如下：

```
import cv2
import numpy as np
o=cv2.imread("image\\lena.bmp")
kernel = np.ones((9,9),np.float32)/81
r = cv2.filter2D(o, -1, kernel)
cv2.imshow("original",o)
cv2.imshow("Gaussian",r)
cv2.waitKey()
cv2.destroyAllWindows()
```

此程序会将原始图像以 9×9 大小的邻域进行均值滤波，程序运行结果如图 7-35 所示。

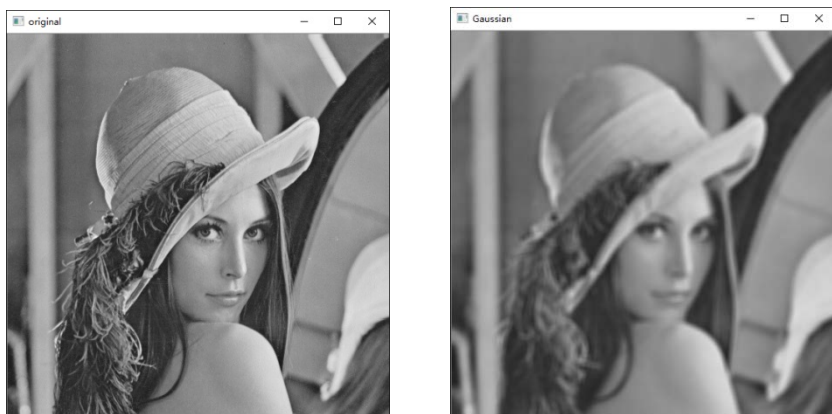


图 7-35 【例 7.10】程序的运行结果

当然，本例中使用的卷积核比较简单，该滤波操作与直接使用均值滤波语句“`r=cv2.blur(o,(5,5))`”的效果是一样的。在实际应用中，可以定义更复杂的卷积核实现自定义滤波操作。